
Desafío Técnico: "Ta-Te-Ti Real-Time Arena"

Objetivo

Desarrollar una aplicación web multijugador del clásico juego Ta-Te-Ti (Tic-Tac-Toe) que permita a los usuarios autenticarse mediante Google, ver su historial de partidas y jugar en tiempo real contra otro oponente.

1. Requerimientos Funcionales

A. Autenticación (SSO & JWT)

- El usuario debe ingresar mediante un botón "Sign in with Google".
- **Flujo de Registro/Login:**
 1. El Frontend obtiene el token de Google.
 2. Envía el token al Backend.
 3. El Backend valida el token con Google.
 4. **Si el email existe:** Se inicia sesión y se retorna un JWT propio de la app.
 5. **Si el email NO existe:** Se crea el usuario en la base de datos (Email, Nombre, Avatar) y luego se retorna el JWT.

B. Dashboard (Lobby)

- Mostrar datos del perfil (Nombre y Avatar de Google).
- Mostrar estadísticas: Partidas jugadas, Ganadas, Perdidas, Empatadas.
- Botón para **"Buscar Partida"**.

C. El Juego (Multijugador)

- Al buscar partida, el sistema debe emparejar a dos usuarios conectados mediante **Socket.IO**.
 - Si no hay oponentes, el usuario queda en estado "Esperando".
 - La partida debe sincronizar los turnos en tiempo real.
 - Al finalizar (ganador o empate), se debe actualizar la base de datos y reflejar el resultado en ambas pantallas.

2. Requerimientos Técnicos (Stack Obligatorio)

Frontend

- ReactJS.
- **Estado Global: Zustand.** (Debe usarse para manejar la sesión del usuario y el estado del tablero en tiempo real).
- **UI:** Material UI (MUI). Diseño limpio y responsivo.
- **HTTP:** Axios (con interceptors para inyectar el JWT en los headers).
- **Comunicación:** Socket.IO Client.

Backend (Elegir uno)

El candidato puede optar por cualquiera de estas opciones, pero debe respetar el uso de WebSockets/Socket.IO:

- **.NET (C#):** ASP.NET Core Web API. (Nota: Se puede usar SignalR como nativo, o una librería de Socket.IO para .NET).
- **Python:** FastAPI o Flask (usando python-socketio).
- **JAVA:** Spring Boot (usando netty-socketio o Spring Websockets).

Base de Datos

- Libre elección (PostgreSQL, Microsoft SQLServer o MongoDB). Se valora el uso de un ORM (Entity Framework, TypeORM, Hibernate, SQLAlchemy, etc.). Plus, utilizar una base de datos como servicio de las gratuitas que existen.

3. Criterios de Evaluación (Lo que vas a mirar)

Para un perfil SSR, no solo buscamos que "funcione", sino que esté bien construido:

Criterio	Solución funcionando, commits claros y decisiones tomadas.
Arquitectura	Separación clara de responsabilidades (Controladores, Servicios, Repositorios). Estructura de carpetas ordenada en el Front.
Manejo de Estado	Uso correcto de Zustand . No debe haber "prop drilling" excesivo. El store debe manejar la lógica de conexión del socket.
Seguridad	El JWT debe validarse en los

	endpoints privados y en la conexión del Socket. No guardar secretos en el frontend.
Código Limpio	Nombres de variables descriptivos, funciones pequeñas, uso de async/await correcto.
Manejo de Errores	Feedback visual si falla el login o se desconecta el socket (Toast notifications o Alertas de MUI).

4. Puntos Bonus (Opcionales)

Si el candidato termina rápido, puede implementar uno de estos para sumar puntos:

- **Reconexión:** Si un usuario refresca el navegador en medio de la partida, el sistema lo vuelve a conectar a la misma sala.
- **Chat:** Un chat simple dentro de la partida.
- **Docker:** Entregar el proyecto con un docker-compose listo para levantar front, back y base de datos.

5. Sugerencia de Entregable (Readme)

Te sugiero enviarles este formato de JSON para que sepan qué estructura de datos esperas en el Frontend para el historial, y así estandarizar la prueba:

JSON

```
// Ejemplo de respuesta esperada en el endpoint GET /profile/stats
{
  "username": "DevCandidate",
  "wins": 12,
  "losses": 5,
  "draws": 2,
  "match_history": [
    {
```

```
"opponent": "Jugador2",  
"result": "WIN",  
"date": "2023-10-27T10:00:00Z"  
}  
]  
}
```